객체와 클래스

객체

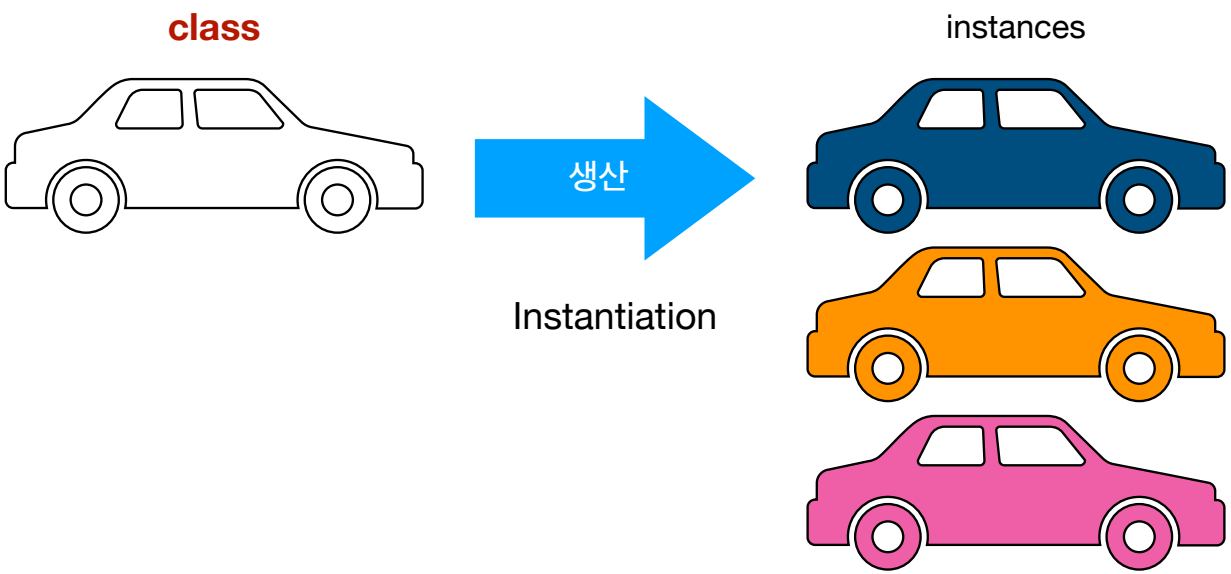
- 객체 object
  - 이름을 붙인 모든 것(소프트웨어 세계에 구현할 **대상**)
  - 데이터(속성 attribute)와 그 데이터에 관련된 함수(동작 method/operator)를 묶어서 관리하는 단위

속성?

동작?

클래스

- 클래스 class
  - 객체의 속성과 행동을 정의한 설계도



인스턴스

- 인스턴스 instance
  - 클래스를 바탕으로 소프트웨어 세계에 **구현된 구체적 실체**

- 객체

- 클래스

```
class Car:
    def __init__(self, maker = "", model = "", year = -1, price =-1):
        self.maker = maker
        self.model = model
        self.year = year
        self.price = price

    def drive(self):
        print("drive")

    def turn(self):
        print("turn")

    def park(self):
        print("park")
```

- 인스턴스

```
mycar = Car()
mycar.drive()

yourcar = Car()
yourcar.drive()
```

클래스 멤버와 인스턴스 멤버

- 클래스 멤버
  - class name space에 생성
  - 모든 객체에서 공유 사용
- 인스턴스 멤버
  - instance name space에 생성
  - 개체별로 각각 사용

PYTHON으로 말하기

```
1.   class foo:
2.       class_name_space_var = 100                  # 클래스 멤버

4.       def set_instance_name_space_var(self, x):
5.           self.instance_name_space_var = x        # 인스턴스 멤버

7.       def get_instance_name_space_var(self):
8.           return self.instance_name_space_var

10.  obj1 = foo()
11.  obj1.set_instance_name_space_var(10)

13.  obj2 = foo()
14.  obj2.set_instance_name_space_var(20)

16.  print(dir(foo))      #1.
17.  print(dir(obj1))     #2.
18.  print(dir(obj2))     #3.

20.  print('obj1:{0}, {1}'.format(obj1.get_instance_name_space_var(), obj1.class_name_space_var))     #4.
21.  print('obj2:{0}, {1}'.format(obj2.get_instance_name_space_var(), obj2.class_name_space_var))     #5.
```

RUN

```
1.      ['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__init__',
         '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
                '__str__', '__subclasshook__', '__weakref__', 'class_name_space_var', 'get_instance_name_space_var', 'set_instance_name_space_var']

2.      ['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__init__',
         '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
                '__str__', '__subclasshook__', '__weakref__', 'class_name_space_var', 'get_instance_name_space_var', 'instance_name_space_var',
                                                                                 'set_instance_name_space_var']

3.      ['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getattribute__', '__gt__', '__hash__', '__init__',
         '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
                '__str__', '__subclasshook__', '__weakref__', 'class_name_space_var', 'get_instance_name_space_var', 'instance_name_space_var',
                                                                                 'set_instance_name_space_var']

4.                                                                                 obj1:10, 100

5.                                                                                 obj2:20, 100
```

클래스

- 객체
  - 데이터와 그 데이터에 관련된 함수를 묶어서 관리하는 단위
  - 이름을 붙인 모든 것
- 클래스
  - 객체의 구조와 행동을 정의한 설계도
  - 클래스를 정의하기

colon

colon

```
class 클래스 이름:
    def __init__(self, param):
        
```

self

- (파이썬)클래스 메소드와 일반함수의 차이점
- Java / C++ 의 this 와 같은 개념
- 메소드 호출시 파이썬이 자동으로 값을 할당

```
class Robot:

    def sayHi(self):
        print("Hello, how are you?")


Robot().sayHi()          # 클래스 메소드 호출

r = Robot()              # 객체.메소드 호출
r.sayHi()

Robot.sayHi(r)           # 클래스.메소드 호출, 반드시 self에 객체 인스턴스 지정
```

[예제] self 사용법

```
class Robot:

    def setname(self, name):
        self.name = name

    def whoami(self):
        print("My name is {}".format(self.name))

    def sayHi(self):
        print("Hello, how are you?")


>>> r = Robot()
>>> r.setname("C-3PO")

>>> r.whoami()
```

[예제] self 사용법#1

```
class Robot:

    def setname(self, name):
        self.name = name

    def whoami(self):
        print("My name is {}".format(self.name))

    def say_hi(self):
        print("Hello, how are you?")


>>> r = Robot()
>>> #r.setname("C-3PO")

>>> r.whoami()
```

self.name이 존재하지 않으므로 에러 발생
예제의 name은 어디에서 작업하는 것이 좋을까 ???

생성자와 초기화자

- 생성자로 객체생성 호출 순서
  - `__new__`
  - `__init__`

```
class Robot:

    def __new__(cls):
        return super().__new__(cls)

    def __init__(self):
        super().__init__()


>>> r = Robot() # 객체생성을 위한 생성자 호출
```

[예제] 생성자와 초기화자에 인자 넣기

```
class Robot:

    def __new__(cls, name):
        return super().__new__(cls)

    def __init__(self, name):
       self._name = name
        super().__init__()

    def setname(self, name):
        self.name = name

    def whoami(self):
        print("My name is {}".format(self._name))

    def sayHi(self):
        print("Hello, how are you?")


r = Robot("C-3PO") # 객체생성을 위한 생성자 호출
r.whoami()
```

클래스 변수와 객체 변수

- 클래스 변수
  - (해당 클래스 로부터) 생성된 모든 인스턴스 사이에 공유됨

- 객체 변수
  - (해당 클래스 로부터) 생성된 각각의 객체/인스턴스에 속한 변수

[예제] Robot.py

```
class Robot:
    """이름을 갖고 있는 로봇을 작성합니다."""

    # 클래스 변수, 로봇 인스턴스를 세는 변수
    population = 0

    def __init__(self, name):
        """객체 변수 초기화"""
        self._name = name

        # 로봇 객체가 생성될 때, 객체 수를 1 증가 시킨다.
        Robot.population += 1

    def die(self):
        """소멸자"""

        print ("{}가 소멸됩니다.".format(self._name))
        Robot.population -= 1
        if Robot.population == 0:
            print("{}가 마지막 로봇 이였습니다.".format(self._name))
        else:
            print("이제 {:d}대의 로봇이 남아 있습니다.".format(Robot.population))

    def sayHi(self):
        print("안녕하세요. 주인님 저는 {} 입니다.".format(self._name))


    @classmethod
    def count(cls):
        print("로봇이 모두 {}대 있습니다.".format(cls.population))
```

[예제] Robot.py

```python
class Robot:
    """이름을 갖고 있는 로봇을 작성합니다."""

    # 클래스 변수, 로봇 인스턴스를 세는 변수
    population = 0

    def __init__(self, name):
        """객체 변수 초기화"""
        self.name = name

        # 로봇 객체가 생성될 때, 객체 수를 1 증가 시킨다.
        Robot.population += 1

    def die(self):
        """소멸자"""
        print ("{}가 소멸됩니다.".format(self.name))
        Robot.population -= 1
        if Robot.population == 0:
            print("{}가 마지막 로봇 이였습니다.".format(self.name))
        else:
            print("이제 {:d}대의 로봇이 남아 있습니다.".format(Robot.population))

    def sayHi(self):
        print("안녕하세요. 주인님 저는 {} 입니다.".format(self.name))

    @classmethod
    def count(cls):
        print("로봇이 모두 {}대 있습니다.".format(cls.population))

    @staticmethod
    def print_msg(msg):
        print('메시지 "{}"가 도착했습니다.'.format(msg))
```

```python
droid1 = Robot("R2D2")
droid1.sayHi()
Robot.count()

droid2 = Robot("C-3PO")
droid2.sayHi()
Robot.count()

droid1.die()
droid2.die()
Robot.count()
Robot.print_msg("zzz")
```

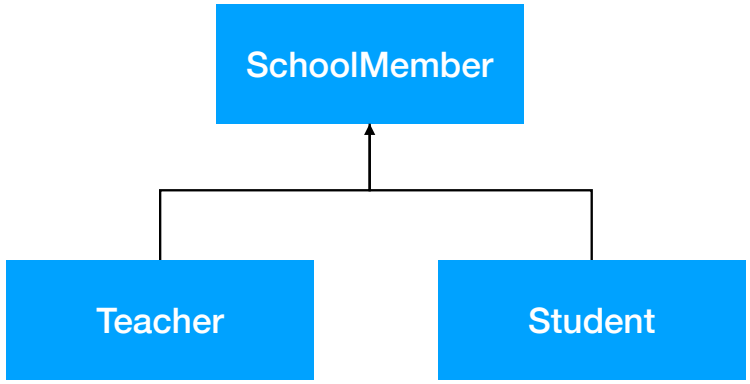
MyCar.py VS. MyCar.java

```python
class MyCar:

    engine = 0                          # 클래스 변수

    def __init__(self, speed, tire):    # 생성자
        self.speed = speed              # self.* 은 인스턴스 변수
        self.tire = tire
        MyCar.engine += 1


    @static-method
    def is_engine(speed1, speed2):
        return speed1 == speed2

    @class-method
    def get_engine(cls):
        return cls.engine

my_car = MyCar( 5, '한국')

print(my_car.speed)
print(my_car.tire)
print(my_car.get_engine())
```

```java
class MyCar{

    private static int engine = 0;
    private int speed;
    private String tire;

    public MyCar(int speed, String tire) {
        this.speed = speed;
        this.tire = tire;
        MyCar.engine += 1;
    }

    public void setSpeed(int speed) {
        this.speed = speed;
    }

    public int getSpeed() {
        return speed;
    }

    public void setTire(String tire) {
        this.tire = tire;
    }

    public String getTire() {
        return tire;
    }

    public static int getEngine() {
        returen MyCar.engine;
    }

    public static void main(String[] args) {
        MyCar myCar = new MyCar(5, "한국");

        System.out.println(myCar.getSpeed());
        System.out.println(myCar.getTire());
        System.out.println(myCar.getEngine());
    }
}
```

상속

- 코드 재사용의 한 방법
- 클래스 간의 형식과 세부 형식을 구현



[예제] SchoolMemeber.py

```
class SchoolMember:
    """Represents any school memeber."""
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print("(Initialized SchoolMember: {})".format(self.name))

    def tell(self):
        '''Tel my details,'''
        print('Name:"{}" Age:"{}"'.format(self.name, self.age), end = ' ')

class Teacher(SchoolMember):
    '''Represents a teacher.'''
    def __init__(self, name, age, salary):
        SchoolMember.__init__(self, name, age)
        self.salary = salary
        print('(Initialized Teacher: {})'.format(self.name))

    def tell(self):
        SchoolMember.tell(self)
        print('Salary: "{:d}"'.format(self.salary))

class Student(SchoolMember):
    '''Represents a student.'''
    def __init__(self, name, age, marks):
        SchoolMember.__init__(self, name, age)
        self.marks = marks
        print('(Initialized Student: {})'.format(self.name))

    def tell(self):
        SchoolMember.tell(self)
        print('Marks: "{:d}"'.format(self.marks))
```

[예제] SchoolMemeber.py

```
class SchoolMember:
    """Represents any school memeber."""
    def __init__(self, name, age):
        self.name = name
        self.age = age
        print("(Initialized SchoolMember: {})".format(self.name))

    def tell(self):
        '''Tel my details,'''
        print('Name:"{}" Age:"{}"'.format(self.name, self.age), end = ' ')

class Teacher(SchoolMember):
    '''Represents a teacher.'''
    def __init__(self, name, age, salary):
        SchoolMember.__init__(self, name, age)
        self.salary = salary
        print('(Initialized Teacher: {})'.format(self.name))

    def tell(self):
        SchoolMember.tell(self)
        print('Salary: "{:d}"'.format(self.salary))

class Student(SchoolMember):
    '''Represents a student.'''
    def __init__(self, name, age, marks):
        SchoolMember.__init__(self, name, age)
        self.marks = marks
        print('(Initialized Student: {})'.format(self.name))

    def tell(self):
        SchoolMember.tell(self)
        print('Marks: "{:d}"'.format(self.marks))
```

```
t = Teacher("Mrs. Sullivan", 57, 30000)
s = Student("Keller", 20, 75)

members = [t, s]
for member in members:
    member.tell()
```

객체와 클래스 스터디: 거북이 보물찾기 게임

참고: 한선관, 김태령, “Chapter 13. 객체와 클래스 프로젝트”, 컴퓨팅 사고를 위한 파이썬, 생능출판

게임 시나리오

- 입력

  1. 초록 거북(주인공)의 이동속도를 입력 받는다.
  2. 적 거북의 숫자를 입력 받는다.
  3. 적 거북의 이동 속도를 입력 받는다.
  4. 보물(사각형)의 개수를 입력 받는다.

- 처리

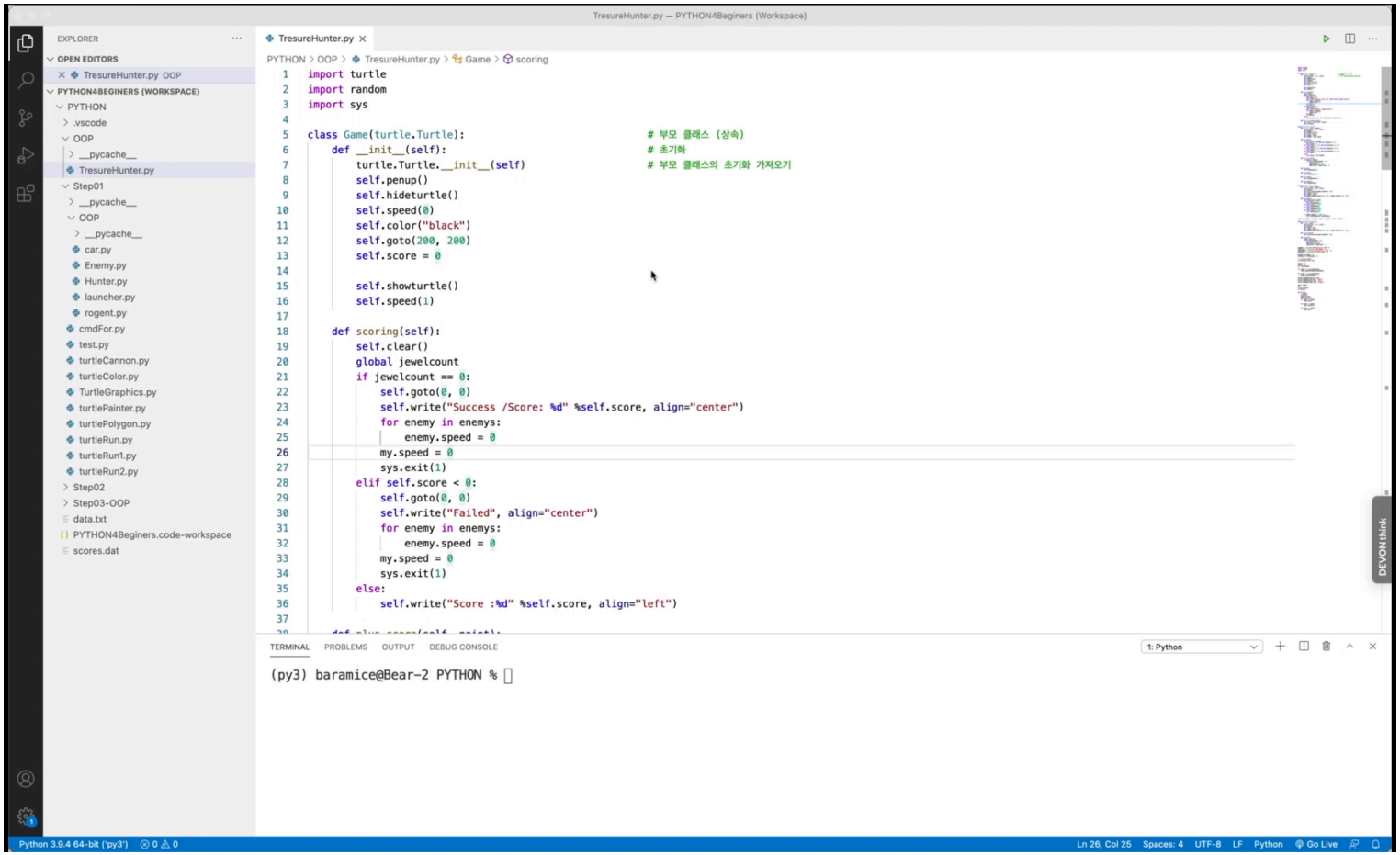
  1. 초록 거북이 돌아다니면서 보물을 얻는다.
  2. 초록 거북이 보물을 얻으면 10점씩 증가한다.
  3. 적 거북은 무작위로 돌아다니면서 초록 거북을 위협한다.
  4. 적 거북과 초록 거북이 충돌하면 10점씩 감소한다.

- 출력

  1. 보물을 다 얻으면 화면에 Success 라고 출력하고 종료한다.
  2. 만약 점수가 0점보다 낮아지면 Failed 라고 출력하고 종료한다.

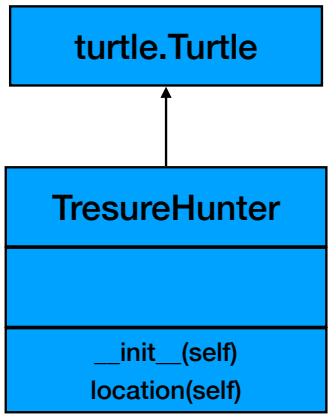
게임 시나리오

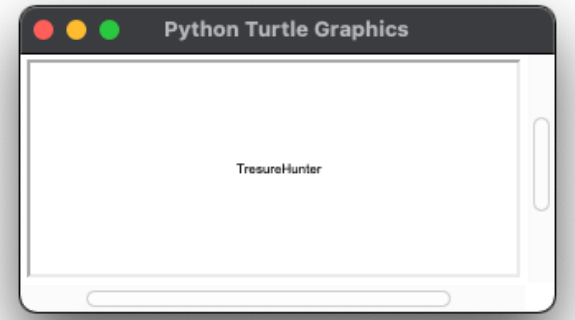
- 실행화면


TresureHunter.py — PYTHON4Beginners (Workspace)
EXPLORER
OPEN EDITORS
TresureHunter.py OOP
PYTHON4BEGINERS (WORKSPACE)
PYTHON
.vscode
OOP
__pycache__
TresureHunter.py
Step01
__pycache__
OOP
__pycache__
car.py
Enemy.py
Hunter.py
launcher.py
rogent.py
cmdFor.py
test.py
turtleCannon.py
turtleColor.py
TurtleGraphics.py
turtlePainter.py
turtlePolygon.py
turtleRun.py
turtleRun1.py
turtleRun2.py
Step02
Step03-OOP
data.txt
PYTHON4Beginers.code-workspace
scores.dat
TresureHunter.py
PYTHON > OOP > TresureHunter.py > Game > scoring
import turtle
import random
import sys
class Game(turtle.Turtle): # 부모 클래스 (상속)
    def __init__(self): # 초기화
        turtle.Turtle.__init__(self) # 부모 클래스의 초기화 가져오기
        self.penup()
        self.hideturtle()
        self.speed(0)
        self.color("black")
        self.goto(200, 200)
        self.score = 0
        self.showturtle()
        self.speed(1)
    def scoring(self):
        self.clear()
        global jewelcount
        if jewelcount == 0:
            self.goto(0, 0)
            self.write("Success /Score: %d" %self.score, align="center")
            for enemy in enemys:
                enemy.speed = 0
            my.speed = 0
            sys.exit(1)
        elif self.score < 0:
            self.goto(0, 0)
            self.write("Failed", align="center")
            for enemy in enemys:
                enemy.speed = 0
            my.speed = 0
            sys.exit(1)
        else:
            self.write("Score :%d" %self.score, align="left")
TERMINAL
PROBLEMS
OUTPUT
DEBUG CONSOLE
1: Python
(py3) baramice@Bear-2 PYTHON %
DEVONthink
Python 3.9.4 64-bit ('py3')
Ln 26, Col 25
Spaces: 4
UTF-8
LF
Python
Go Live

클래스 정의와 객체 생성

```
import turtle

class TresureHunter(turtle.Turtle):
    def __init__(self):
        turtle.Turtle.__init__(self)
        self.penup()
        self.hideturtle()
        self.goto(0,0)
        self.write("TresureHunter", align="center")

    def location(self):
        h = self.heading()
        x = self.xcor()
        y = self.ycor()
        dis = self.distance(self)

        print("Location of TresureHunter: ")
        print("X Coor:", x, ", Y Coor:", y, ", Direction:", h)
        print("Distance:", dis)

a = TresureHunter()
a.location()

turtle.mainloop()
```





클래스 정의와 객체 생성

- 여러 객체 생성

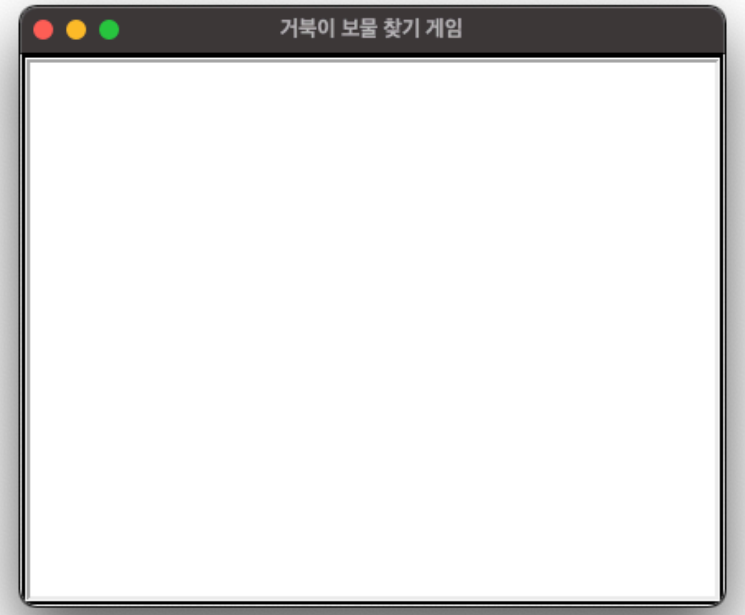
```
import turtle

class TresureHunter(turtle.Turtle):                    # 부모 클래스 (상속)
    def __init__(self):                                # 초기화
        turtle.Turtle.__init__(self)                   # 부모 클래스의 초기화 가져오기
        self.penup()
        self.goto(0,0)

        self.showturtle()
        self.speed(1)

    def location(self):""
```

```
turtles = []                                    # list()
for a in range(5):                              # 5번 반복
    turtles.append(TresureHunter())             # TresureHunter 거북이를 리스트에 객체 넣기

for a in turtles:                               # 객체 순회 interator
    a.fd(100)                                   # 하나씩 100 전진
```

게임 루프

- turtle.Screen()

```
import turtle

s = turtle.Screen()
s.title("거북이 보물 찾기 게임")

s.tracer(0)                          # 화면을 빠르게 업데이트
turtle.listen()

while True:                          # 게임 메인 루프
    s.update()
```



보물 찾는 거북이(TresureHunter)

- Hunter 클래스
  - 주인공 클래스(my)
  - 펜 올리기, 거북 모양, 초록색
  - 키보드 상하좌우 화살표

```
import turtle

class Hunter(turtle.Turtle):
    def __init__(self, speed=1):
        turtle.Turtle.__init__(self)
        self.penup()
        self.shape("turtle")
        self.color("green")
        self.speed = speed

    def up(self):
        self.setheading(90)

    def down(self):
        self.setheading(270)

    def left(self):
        self.setheading(180)

    def right(self):
        self.setheading(0)

    def move(self):
        self.forward(self.speed)
```

보물 찾는 거북이(TresureHunter)

- Hunter 클래스
- 게임 루프

```python
import turtle

class Hunter(turtle.Turtle):
    def __init__(self, speed=1):
        turtle.Turtle.__init__(self)
        self.penup()
        self.shape("turtle")
        self.color("green")
        self.speed = speed

    def up(self):
        self.setheading(90)

    def down(self):
        self.setheading(270)

    def left(self):
        self.setheading(180)

    def right(self):
        self.setheading(0)

    def move(self):
        self.forward(self.speed)
```

```python
import turtle

s = turtle.Screen()
s.title("거북이 보물 찾기 게임")

my = Hunter(0.1)

turtle.onkeypress(my.up, “Up”)
turtle.onkeypress(my.down, “Down”)
turtle.onkeypress(my.left, “Left”)
turtle.onkeypress(my.right, “Right”)

s.tracer(0)
turtle.listen()

while True:
    s.update()
    my.move()
```

보물을 지키는 거북이(Enemy)

- Enemy 클래스
  - 적 클래스(enemy)
  - 다양한 색의 거북이, 첫 위치 랜덤
  - 지정된 속도만큼 앞으로 이동

```
import turtle
import random

colors = ["beige", "yellow", "pink", "orange", "red", "black"]  # 적 거북 컬러 선택지

class Enemy(turtle.Turtle):
    def __init__(self, speed=1):
        turtle.Turtle.__init__(self)
        self.penup()
        self.shape("turtle")
        self.speed = speed
        self.color(colors[random.randint(0, 5)])                # 다양한 색
        self.goto(random.randint(-250, 250), random.randint(-250, 250))

    def move(self):
        self.forward(self.speed)
```

보물을 지키는 거북이

- Hunter 클래스

```
colors = ["beige", "yellow", "pink", "orange", "red", "black"]

class Enemy(turtle.Turtle):
    def __init__(self, speed=1):
        turtle.Turtle.__init__(self)
        self.penup()
        self.shape("turtle")
        self.speed = speed
        self.color(colors[random.randint(0, 5)])
        self.goto(random.randint(-250, 250), random.randint(-250, 250))


    def move(self):
        self.forward(self.speed)
        # 0.1% 확률로 주인공을 따라온다.
        if random.randint(1, 1000) == 1:
            self.setheading(self.toward())
```

- 게임 루프

```
enemy(0.2)

while True:                          # 게임 메인 루프
    enemy.move()
```

보물을 지키는 거북이

- 벽 충돌 방지하기 Enemy.move()
  - x 좌표가 250을 넘어가면 거북이 머리를 반대 방향으로 돌린다. 각 네 방향에 대하여 똑같이 적용한다.

```
    def move(self):
        self.forward(self.speed)
        if self.xcor() > 250:
            self.setheading(180)
        elif self.xcor() < -250:
            self.setheading(0)
        elif self.xcor() > 250:
            self.setheading(270)
        elif self.ycor() < -250:
            self.setheading(90)

        if random.randint(1, 1000) == 1:
            self.setheading(self.towards(my))
```

보물을 지키는 거북이

- 벽 충돌 방지하기 Enemy.move()
  - 좌표를 넘어가면서도 해당 방향을 계속 보고 있으면 속도 0, 다른 방향이면 회복

```
    def move(self):
        self.forward(self.speed)
        if self.xcor() > 250 and self.heading() == 0:
            self.speed = 0
        elif self.xcor() < -250 and self.heading() == 180:
            self.speed = 0
        elif self.ycor() > 250 and self.heading() == 90:
            self.speed = 0
        elif self.ycor() < -250 and self.heading() == 270:
            self.speed = 0
        else:
            self.speed = self.speed2
```

보물(보석)

- Jewel 클래스
  - 네모난 보석
  - 여러 색을 이용하여 보석 색이 계속 바뀜
  - 보물 찾는 거북이 매우 가까이 다가오면, 보석이 화면 바깥으로 이동

```
class Jewel(turtle.Turtle):
    def __init__(self):
        turtle.Turtle.__init__(self)
        self.penup()
        self.color("red")
        self.shape("square")
        self.goto(random.randint(-250, 250), random.randint(-250, 250))

    # 반짝이게 만들기
    def twinkle(self):
        self.color(colors[random.randint(0, 5)])    # colors 리스트 이용

jewel = Jewel()

while True:                             # 게임 메인 루프
    jewel.twinkle()
```

게임 만들기

- 점수, 게임 진행 클래스
  - 거북이를 생성해 해당 위치에 배치한다.
  - 점수를 오른쪽 상단에 기재한다.
    - 만약 점수가 0보다 작아면 모든 거북들이 멈추고 난 후 화면 중간에서 Failed를 출력
    - 만약 점수가 10점이 되면 화면 가운데 Success를 출력
  - 점수 변수 및 +10 더하는 메서드

게임 만들기

- 점수, 게임 진행 클래스
- 게임 루프

```
class Game(turtle.Turtle):
    def __init__(self):
        turtle.Turtle.__init__(self)
        self.penup()
        self.hideturtle()
        self.speed(0)
        self.color("black")
        self.goto(200, 200)
        self.score = 0

    def scoring(self):
        self.clear()
        if jewelcount == 0:
            self.goto(0, 0)
            self.write("Success /Score: %d" %self.score, align="center")
            enemy.speed = 0
            my.speed = 0
        elif self.score < 0:
            self.goto(0, 0)
            self.write("Failed", align="center")
            enemy.speed = 0
            my.speed = 0
        else:
            self.write("Score :%d" %self.score, align="left")

    def plus_score(self, point):
        self.score = self.score + point

game = Game()               # 객체 생성

while True:                 # 게임 메인 루프
    game.scoring()
```

적 거북이 / 보물 충돌 이벤트

- 점수 기능
  - 상승 조건: 보석에 닿으면 10점이 증가한다. 닿은 보석은 사라진다.
  - 하락 조건: 적 거북에 닿으면 10점이 감소한다. 닿은 적 거북은 사라진다.

```
# Jewel 클래스
    def eat(self):

# Hunter 클래스
    def collision(self):

# 게임 메인 루프
while True:
    my.collision()
    jewel.eat()
```

[과제] 게임 기능 확장하기

- 게임 종료

  sys 모듈

  게임 종료 시 sys.exit(1)로 게임 루프 종료

- 스크린 배경 그림 넣기

```
s=turtle.Screen()
s.bgpic("배경 그림 파일")
```

- 캐릭터, 적 거북, 보석 이미지 바꾸기

  image = "이미지 파일"

  s = turtle.Screen()

  s.addShape(image)

  turtle.shape(image)

- 효과음 넣기(따로 재생)

```
import os
os.system('file_path/file_name.mp3')
```